

Tarea A

Ejecuta cada uno de los comandos equivalentes en ambos sistemas.

- Escribe las diferencias en el formato de salida.
- Explica cuál te resulta más intuitivo y por qué.

Listar contenido de un directorio

Bash (ls -l): Muestra permisos, propietario, grupo, tamaño y fecha en columnas.

PowerShell (Get-ChildItem): Muestra una tabla con columnas Mode, LastWriteTime, Length, y Name.

La salida de Bash tiene menos datos así que supongo que para usarlo en programas o scripts será más útil.

Cambiar de carpeta

Bash cd nombre_carpeta y en PowerShell Set-Location nombre_carpeta.

Diferencia:

- PowerShell tiene una sintaxis más “verbal”, mientras que Bash es más corto.
- En PowerShell también puedes usar el alias cd, así que son casi equivalentes.

Crear una carpeta

Bash mkdir proyecto y en PowerShell New-Item -ItemType Directory -Name Proyecto.

Diferencias:

PowerShell requiere especificar el tipo de ítem (-ItemType Directory), porque el comando New-Item también puede crear archivos, claves de registro, etc.

En cambio, Bash tiene un comando específico (mkdir).

Comentario: PowerShell es más flexible, pero Bash es más directo.

Copiar archivos

cp origen destino y en PowerShell Copy-Item origen destino.

Diferencias:

- PowerShell sigue una sintaxis más parecida al inglés (“Copy-Item” = “copiar elemento”).
- En Bash es más corto y rápido de escribir.

Mover archivos

Bash mv origen destino y PowerShell Move-Item origen destino.

Diferencias:

Misma lógica que el anterior: PowerShell usa nombres descriptivos; Bash usa abreviaturas.

Borrar archivos

Bash rm archivo y en PowerShell Remove-Item archivo.

Diferencias:

PowerShell puede borrar más tipos de elementos (archivos, carpetas, claves del registro).

Bash rm es más corto, pero peligroso si no se usa con cuidado (rm -rf).

Ver procesos activos

Bash ps y en PowerShell Get-Process.

Diferencias en salida:

Bash (ps): lista PID, TTY, tiempo de CPU, y comando.

PowerShell (Get-Process): tabla con columnas como Handles, NPM(K), PM(K), WS(K), CPU, Id, ProcessName.

Comentario: PowerShell muestra más información por defecto y con formato de tabla.

Filtrar resultados

Bash ls | grep "txt" y en PowerShell Get-ChildItem | Where-Object { \$_.Name -match "txt" }.

Diferencias:

En Bash, los comandos se combinan con | (pipe) y grep busca texto plano.

En PowerShell, se filtran objetos, no texto, usando expresiones lógicas.

Conclusión: ¿Cuál es más intuitivo?

Depende del enfoque:

- Bash (Linux) es más simple y rápido para usuarios técnicos o administradores que prefieren escribir menos y usar pipes de texto.

- PowerShell (Windows) es más estructurado y lógico en lenguaje natural, ideal para automatización y administración de sistemas complejos, ya que trabaja con objetos en lugar de texto.

Personalmente: Me resulta más intuitivo PowerShell, porque sus comandos son autodescriptivos (Get-Process, Copy-Item, etc.) y es fácil entender qué hace cada uno sin memorizar abreviaturas.

Tarea B

Crea una variable de entorno personalizada en ambos sistemas y comprueba si persiste tras cerrar la sesión.

En Bash (Linux)

Variable temporal:

```
export MI_VARIABLE="HolaLinux"
```

```
echo $MI_VARIABLE
```

Al cerrar la sesión, no persiste.

Variable persistente:

Editar el archivo ~/.bashrc y añadir:

```
export MI_VARIABLE="HolaLinuxPersistente"
```

Guardar y recargar con:

```
source ~/.bashrc
```

Ahora sí, persiste tras reiniciar sesión.

En PowerShell (Windows)

Variable temporal:

```
$env:MI_VARIABLE = "HolaWindows"
```

```
echo $env:MI_VARIABLE
```

Al cerrar la sesión, no persiste.

Variable persistente:

```
[System.Environment]::SetEnvironmentVariable("MI_VARIABLE", "HolaWindowsPersistente",  
"User")
```

Comprobar:

```
echo $env:MI_VARIABLE
```

La variable sigue existiendo.

Tarea C

1. Crea un script en bash y su versión equivalente en PowerShell.
2. Añade una estructura condicional (por ejemplo, si el nombre coincide con “admin”, muestra un mensaje distinto).

Script en Bash (script_bash.sh):

```
#!/bin/bash  
echo "Introduce tu nombre:"  
read nombre  
if [ "$nombre" == "admin" ]; then  
    echo "Bienvenido, administrador del sistema."  
else  
    echo "Hola, $nombre. No tienes privilegios de administrador."  
fi
```

Script en PowerShell (script_powershell.ps1):

```
Write-Host "Introduce tu nombre:"  
$name = Read-Host  
if ($name -eq "admin") {  
    Write-Host "Bienvenido, administrador del sistema."  
}  
else {  
    Write-Host "Hola, $name. No tienes privilegios de administrador."  
}
```

Ejemplo de salida:

Introduce tu nombre:

admin

Bienvenido, administrador del sistema.